TeamWeaver 과제 결과 보고서

기술 적합·협업 이력·월별 가용률·등급 정원·월 예산을 한 번에 계산해 프로젝트 인력 배치 안을 내고, 그 근거를 검사받는 AI 글로 설명하는 의사결정 보조 시스템

1. 과제 개요

1.1 배경과 문제

SI 사업을 하는 조직에서는 인력이 사업(프로젝트) 단위로 투입되고, 사업이 끝나면 다른 사업으로 옮겨 간다. 프로젝트 인력 배치를 정하려면 **기술 적합**만이 아니라 사람마다 **달마다 남은 시간(가용률)**, 사업의 **등급별 정원**과 **월 예산**, **함께 일해 본 동료(협업 이력)**를 동시에 맞춰야 한다. 한 조건을 맞추면 다른 조건이 깨지기 쉬워, 담당자의 수작업 프로젝트 인력 배치로는 ① 조건 위반 여부, ② 더 나은 배치가 있었는지, ③ 인사팀에 "왜 이 팀인가"를 설명할 근거를 확인하기 어렵다. 상황은 셋이다 — 가장 잦은 **신규 사업 편성**(대부분 이미 일하는 중, 쉬는·끝나 가는 사람과 몇 명만 옮겨 새 사업 팀을 짬), 진행 사업에 이슈가 생긴 **긴급 투입**, 드문 **연간 계획**(백지에서 6~12개월 프로젝트 인력 배치).

1.2 목표와 달성

목표	달성 기준	결과
G1 모든 조건을 지키는 배치를 계산으로 낸다	100·200·300명에서 빈자리·제약 위반 없이 안 A~D, 운영 중 K 비교	**달성** — 12개 안 모두 빈자리 0·위반 0, 운영 중 18행(K 0~5) 모두 최선 증명(3.1·3.2)
G2 계산이 맞음을 시스템이 스스로 보인다	독립 검증을 통과한 해만 출력, 정답 대조, 최적과의 거리 표시	**달성** — 정답 대조 13/13, 모든 반환 해 독립 검증, 증명 상한과의 거리를 화면·PDF에(3.7)
G3 결정 근거를 지어낸 내용 없이 글로 낸다	AI 글이 기계 검사를 통과한 것만 나가고, 넣은 오류를 걸러 냄	**달성** — 소명 글 검사 통과 99.0%(처음 보는 데이터 100%), 주입 오류 13,316건 모두 검출(3.6)

1.3 범위와 전제

- PoC·과제 제출용이다. **실제 인사 자료는 적재하지 않았다**(적재 불가). 입력 형식은 실제 인사 자료의 항목과 규칙을 받아 설계했고, 개발·검증·시연은 같은 형식의 가상 데이터로 했다.
- 업무 규칙은 SI 현업 관행을 그대로 담았다 — 한 사람은 한 사업에 전일 투입(여러 사업 동시 수행은 부서장이 사유와 함께 허용), 평소엔 대부분 배치돼 있어 신규 사업은 쉬는·끝나 가는 인력과 **최대 K명만 옮겨** 꾸린다, 손대면 안 되는 사업·사람 고정, 중급 자리에 초급·고급을 넣는 등급 대체(사유·예산 증액), 해를 걸친 육아휴직, 확정한 한 건만 되돌리는 확정 취소.
- 점수의 계수(협업 0.3·익숙한 사이 감점 0.2·하위 기술 인정 70% 등)는 설계값이다. 결과를 맞추려고 데이터 생성기나 계수를 조정하지 않았고, 실패·중단도 기록에 남겼다.

1.4 가상 데이터 설계

항목	설계
조직·사업	실제 SI 조직의 프로젝트·인력 현황(그룹 구성·사업 수·등급 구성)을 참고해 만든 가상 데이터. 조직형 묶음 100·200·300명(데이터 플랫폼·AI·업무 자동화 실행 그룹), 사업 21·42·62건(대부분 계획 전에 시작한 진행 사업 + 제안·추가 과제), 계획 6개월(연간 묶음 12개월), 등급 4단계·월 단가표. 운영 중 묶음은 같은 사람으로 대부분 배치된 상황(100명: 87명 배치·대기 10·육아휴직 3)
가용률·정원·예산·사업 단계	실제 조직의 인력 운영 현황을 참고한 가정 — 가용률은 대부분 100%, 3% 해를 걸친 12개월 육아휴직, 2% 한 달 휴가(가용률 그대로). 정원은 달·등급마다 일할 수 있는 인원의 85% 이하, 예산 = 정원 × 단가 × 1.0~1.25. 사업은 2024년 운영(내부 2~3명)·2025년 고도화·2026 상반기 시작·운영 사업의 추가 과제(내부 2명·6개월)가 섞이고, 진행 사업 팀원의 업무 이력·경력·동료 평가가 그 사업에 이어진다
이력·평가	과제 수행자가 실제로 작성한 업무 이력과 동료 평가를 바탕으로 실제와 비슷하게 만든 가상 데이터(실제 자료는 적재하지 않음) — 10년 업무 이력, 기술별 경력 개월, 동료 평가(항목 점수 + 글). 평가 글은 아쉬운 점을 항목마다 문장으로 쓰고, 직설형 평가자 8%, 항목 점수와 온도가 어긋난 글 5%를 섞어 "항목이 아니라 글을 읽어야" 맞힐 수 있게 했다
과거 성과(선택)	고객 평가·교체 요청 기록을 모델과 독립된 숨은 규칙(실제 역량·이전 협업·업종 경험·잦은 이동·고객 궁합·운)으로 생성
지킨 원칙	목표 수치를 위해 생성기·계수를 고치지 않는다. 가상 성과로 모델의 효과를 "입증"하지 않는다(자기가 만든 규칙을 다시 찾는 순환이므로)

2. 수행 결과 요약

아래 수치는 모두 실제 SI 조직을 본뜬 **가상 조직 데이터**(100·200·300명, 계획 기간 6개월, 시연 영상은 100명)로 잰 값이다. "배치 점수"는 사람과 사업 요구 기술이 맞는 정도(기술 적합)와 함께 일한 경험·평가(협업)를 합친 점수로, 모든 배치를 같은 기준으로 채점했다.

+5~13%

최적화 엔진이 짠 프로젝트 인력 배치가 같은 시간을 쓴 규칙 기반 자동 배치(휴리스틱)보다 배치 점수가 높다(100·200·300명). 휴리스틱은 100·300명에서 빈자리 1·3석을 남겼고, 최적화는 모든 규모에서 빈자리·규칙 위반 0(3.1)

0.9%

300명 조직 최선안이 이론상 최대치에서 떨어진 정도 — 계산 엔진이 92초 만에 "이보다 0.9% 넘게 좋은 배치는 없다"를 증명했다(3.1)

0석 · +2.8%

신규 사업 편성(100명): 아무도 옮기지 않아도 빈자리 없이 채우고, 진행 중인 다른 사업에서 최대 5명까지 옮기면 배치 품질 +2.8%. 0~5명 모든 경우가 계산상 최적임을 확인(3.2)

100% vs 12%

AI로 효율화 — 동료 평가 글에만 쓴 혹평(시험 글 34건)을 AI는 모두, 체크 항목만 세는 규칙은 4건만 찾았다. 그래서 100명 조직에서 서로 나쁘게 평가한 28쌍이 "궁합 좋은 쌍"으로 보상받던 문제가 0쌍이 됐다(3.5)

99% · 1.00

AI로 효율화 — 인사팀 소명 글이 기계 검사를 통과한 비율 99%(처음 보는 데이터 100%), 발주 자료에서 필요 기술을 찾는 정밀도 1.00(AI 없이 규칙 0.93), 낯선 기술 이름 정리 1위 정답 95%(문자열 비교 27%)(3.3·3.6)

최적 기술 탐색

솔버 3종(CBC·SCIP·HiGHS)을 같은 문제로 비교해 가장 많이 푼 HiGHS를 채택. LLM에게 100명 배치를 직접 맡기면 규칙을 어기거나 최적화 결과의 55% 품질에 그쳐, **배치는 최적화 엔진이, AI는 글 읽기·쓰기를** 맡도록 나눴다(3.4·3.7)

2.1 구현 기능 — 영상에 나오지 않는 것 중심

화면 흐름은 시연 영상이 순서대로 보여 준다 — 입력 검증·낯선 기술 이름 정리(D1), 연간 계획과 안 확정(D2·D4), 소명 글(D3), What-if(D5), 신규 사업 편성(D6), 긴급 투입·상황을 말로(D7), 인사팀 검토 자료·변경 보고서(D8), 300명·조직 기술 지도(D9). 여기에는 영상에 나오지 않거나 지나가는 기능과 확정 규칙만 적는다.

영역	영상 밖 기능 · 규칙
데이터	인사 시스템에서 매일 자동으로 받기(받기에 실패하면 지금 데이터 유지). 인력이 진행 중인 사업에서 쓰는 기술의 경력 개월은 달이 지나면 서비스가 자동으로 더한다. 한 번의 편성(데이터·설정·결과)을 파일 하나로 저장하고 다시 불러오기(불러올 때 지금 규칙으로 다시 검사)
계산	안이 하나 풀릴 때마다 바로 화면에 보이고, 계산 중에 멈출 수 있다. 밤사이 AI 없이 긴급 대기 명단·연간 계획·신규 사업 비교를 미리 계산해 둔다. 관리자 설정(계산 시간·익숙한 사이 기준 등)
확정 규칙	모든 확정은 데이터의 새 버전으로 남아 되돌릴 수 있고, 특정 확정 한 건만 골라 취소할 수도 있다. 새 규칙 위반이 생기거나 고정한 배치를 빼는 확정은 거절한다. 다른 등급으로 자리 채우기·예산 증액·빈자리 그대로 두기는 사유를 적어야 확정된다. 육아휴직 등으로 투입할 수 없는 인력은 사유와 함께 고를 수 없게 보인다
보고·보안	모든 변경을 지우지 않고 기록해 기간을 골라 보고서 PDF로 묶는다. 계산 결과에 위·변조 방지 서명을 붙이고 PDF는 서버가 다시 계산해 찍는다. 로그인, 실제 인사 자료일 때 동료 평가 원문을 화면·외부 AI에 보내지 않는 모드

3. 정량적 성과

3.1 연초 계획(6개월): 프로젝트 인력 배치 품질과 계산 성능

규모	사업	안마다 시간 한도	최선안 배치 점수	이론상 최대치와의 차이(안 4개)	빈자리·위반	안 4개 계산 시간	휴리스틱 → 최적화(배치 점수)
100명	21	30초	59.47	2.5~5.0%	0 · 0	1.3분	56.61 → 59.47 (**+5.1%**, 휴리스틱 빈자리 1석)
200명	42	60초	151.43	1.0~4.1%	0 · 0	3.0분	133.93 → 151.43 (**+13.1%**)
300명	62	90초	228.10	0.9~3.5%	0 · 0	4.2분	205.72 → 228.10 (**+10.9%**, 휴리스틱 빈자리 3석)
100명 12개월(연간)	40	120초	119.49	2.5~4.7%	0 · 0	5.4분	—

- 계산한 안 4개(A~D, 서로 사람 구성이 충분히 다른 대안) 중 배치 점수가 가장 높은 안이 **최선안**이다(네 경우 모두 A안). 배치 점수 = 기술 적합 + 협업 – 오래 붙어 다닌 사이 감점(빈자리 감점 제외). **이론상 최대치**는 계산 엔진이 "어떤 배치도 이 점수를 넘을 수 없다"고 증명한 값이고, 그와의 차이가 작을수록 더 좋은 배치가 남아 있지 않다는 뜻이다 — 200·300명 최선안은 1% 안까지 증명했고(300명 0.9%·92초), 100명은 시간 한도에서 멈춰 2.5%가 남았다. 사람은 한 번에 한 사업에 전일 투입하고(6.1), 계산 엔진 HiGHS를 4갈래로 동시에 돌려 가장 좋은 해를 고른다(6.3).
- 휴리스틱(비교 대상) = 예산·가용률을 지키며 기술과 협업을 보고 자리를 채운 뒤, 최적화와 **같은 시간** 동안 같은 등급 사람끼리 바꿔 보기를 반복하는 규칙 기반 자동 배치. 두 방식을 같은 채점 기준으로 비교해 최적화 엔진을 채택했다.

3.2 신규 사업 편성: 진행 중인 사업에서 최대 K명만 옮길 때

규모(상황)	프로젝트 인력 배치 품질 K(옮길 수 있는 최대 인원) = 0 → 1 → 2 → 3 → 4 → 5	개선(K=5)	빈자리	최적 확인	K별 최대 시간
100명(87명 배치·대기 10·육아휴직 3·신규 제안 2 + 추가 과제 1)	79.78 → 80.49 → 81.12 → 81.45 → 81.76 → 82.01	+2.8%	0	6/6	3.8초
200명(175명 배치·대기 20·육아휴직 5·신규 제안 3 + 추가 과제 1)	135.63 → 136.97 → 137.96 → 138.88 → 139.73 → 140.45	+3.6%	0	6/6	14.1초
300명(262명 배치·대기 30·육아휴직 8·신규 제안 3 + 추가 과제 1)	189.54 → 190.58 → 191.56 → 192.53 → 193.36 → 194.33	+2.5%	0	6/6	33.6초

각 값은 계산 엔진이 "이보다 나은 배치가 없다"를 확인한 최적값이다(허용 오차 0). 가상 데이터에서 신규 사업의 정원은 대기 인력의 등급 구성에 맞춰 정했다 — 회사가 지금 쉬는 인력으로 꾸릴 수 있는 사업에 제안한다는 가정이라, 아무도 옮기지 않아도(K=0) 빈자리가 없다. K를 늘리면 후보가 대기 인력(100명 조직 10명)에서 이미 다른 사업에서 일하는 인력(87명)까지 넓어져, 기술이 더 맞는 사람을 데려올 수 있어 품질이 오른다.

3.3 AI가 바꾼 것 — AI 없이 할 때와 같은 데이터로 비교

AI는 배치를 정하지 않는다(3.4). **사람이 다 읽을 수 없는 글을 읽고, 근거 글을 쓰고, 부서장의 말을 사전 말로 옮기는** 곳에만 쓰고, 같은 일을 AI 없이(규칙·사전·문자열) 할 때와 나란히 쟀다. 판정 기준은 측정 전에 커밋했다.

일	AI 없이	AI를 쓰면	근거
동료 평가 글 읽기	체크 항목만 센다 — 글에만 쓴 혹평 12%(4/34) 검출, 서로 나쁘게 쓴 28쌍이 "궁합 좋은 쌍"으로 보상	글을 읽는다 — **100%**(34/34)·돌려 말한 지적 89%, 그런 쌍 **0**, 팀 협업 점수 +7~22%, 300명 4,012건 $0.32	3.5
"왜 이 팀인가" 글	사실 77줄(4,637자)을 늘어놓거나 사람이 쓴다	세 문장(231자) 요약 + 사실 번호 연결, 검사 통과 **99%**(처음 보는 데이터 100%), 편당 8초·$0.0009	3.6
교체 설명	점수 표만	동료 평가 원문 인용 설명 3.5초, 바로 사용 93%, 위반 교체 "보류" 18/20	3.6
긴급 상황을 말로(새)	사전 이름이 그대로 적힌 기술만 — 재현율 83.6%, 일로만 설명한 요구 **0%**	재현율 **100%**(55/55)·정밀도 93%, 일로 설명한 요구 100%, 사람 특성 말 반영 0, 건당 $0.0005·2.5초	요청 48문장
낯선 기술 이름(새)	문자열 유사도 — 실제 별칭 522개 1위 27%·후보 3개 안 54%, 잘못 합침 30	1위 **95%**·후보 3개 안 **100%**, 잘못 합침 1, 기술 아님·새 기술 20개 합침 0. 정답대로 승인하면 틀린 기술 점수 34.7% → **0**	별칭 숨김 시험
발주 자료 → 필요 기술(새)	부서장이 RFP를 읽고 기술을 하나씩 고름. 규칙(사전 이름 + 문장 말)은 현행 시스템 언급을 요구로 뽑기도 함 — 정밀도 0.93, 경력 83%·인원 77% 일치	정밀도 **1.00**·재현율 0.99, 현행·연계 언급 오검출 **0**, 경력 98%·인원 100%, 편당 6초·$0.001(사내 대리 GLM 정밀도 0.93). 8만 자 자료는 나눠 동시에 읽음. 근거 말과 함께 제안 → 사람이 고침	자료 18편
발주 자료 → 사업 정보·요약(새)	공고문을 보며 사업명·기간·예산을 손으로 적고 자료를 다 읽는다. 규칙은 "사업명:" 같은 표지가 있을 때만 — 가상 7편은 규칙도 7/7(쉬운 세트)	실제 공고문의 계약기간 "계약체결일 ~ 11월 13일"을 **7개월**로 셈(규칙은 못 함)·표지 없는 사업명도 찾음 — 근거로 확인 못 한 값은 "확인 필요". 요약은 자료마다 과업·기간·예산·인력·평가·유의 사항, 근거·숫자 검사로 항목을 거름(완전한 차단은 아님), 편당 5~14초	가상 7편 + 실제 공고 자료

평가 글 한 건(가상) — 체크: 좋은 점 5 · 아쉬운 점 2

"자기 범위만 챙기고 팀 일에는 무관심했습니다. 일정을 계속 어겼습니다. 다시 함께 일정을 잡고 싶지 않습니다."

항목만 세면 +0.43 → "궁합 좋은 상위 200쌍" · AI는 −0.80 → 보상에서 뺌(100명 중 이런 쌍 31개)

같은 사실로 쓴 "왜 이 팀인가"(J001) — AI 없이: 77줄 중 첫 줄

대상 사업: 사업 J001 …(팀 8명: …). [F1] / 요구 기술 메타데이터 관리: 3명 필요(최소 경력 24개월 = 레벨 2), 팀에서 해당자 6명 → 충족. [F2] / …

AI 요약(세 문장 전부)

요구 기술 충족 현황은 메타데이터 관리 … 해당자 6명으로 충족[F2], ETL … 충족[F3]입니다. 규칙 준수 현황은 … 위반 0건 …[F77]입니다. 다만 주성진은 금융 업무 경력 21개월로 레벨 2(요구 레벨 3) 미달[F20]입니다.

새 세 기능은 AI가 말·이름·자료를 사전 개념으로 옮기기만 하고(사람 정보는 보내지 않음), 추천은 기존 계산이, 결정은 사람이 한다. 표본이 작아 구간이 넓다(요청 48문장 정밀도 95% 구간 0.84~0.97). 발주 자료 18편 중 11편은 규칙과 같은 사람이 쓴 가상 자료다. 시연은 나라장터에 공개된 공공 입찰 자료(사전규격 R26BD00208180 · 산안공고 제2026-68호의 공고문·제안요청서)로 했다 — 민간 기업의 RFI·RFP는 초청한 업체에만 보내는 비공개 문서라 구할 수 없다. 이 연구 용역 자료(계약 조건까지 7편)에는 기술별 경력·인원 요건이 없어 AI가 과업 내용에서 기술을 추정하고 화면이 "추정"으로 표시한다(위 정답표 측정과 섞지 않음). 근거 `rehearsal/results/` request-parse·skill-mapping·rfp-read.

E5·E6·E7c(3.4·3.6)는 10-12 데이터(예전 투입 방식) 기준이다 — 외부 AI 비용 때문에 다시 재지 않았다. 3.5의 판정 정확도(E4)는 평가 글이 그대로라 지금도 맞고, 배치 영향은 10-19에 다시 쟀다.

3.4 AI에게 맡기지 않는 곳 — LLM 직접 배치 (E6)

외부 LLM(gpt-6-luna, 추론 high)에게 같은 규칙·목적을 설명하고 배치를 받아 서비스 평가기로 채점했다. 20명·사업 4에서는 솔버의 89~94%였지만, 50명은 원자료만 주면 위반(점수 제공·1회 수정 시 69.6%), 100명은 위반 또는 **솔버의 55%**에 응답 6~11분(솔버 30.6초). 칸마다 2~3회라 표본은 작다. → **배치 결정은 솔버가, AI는 읽기·쓰기를** 맡는 구조의 근거(`rehearsal/results/llm-vs-solver.html`).

3.5 평가 글은 AI가 읽어야 한다 — 판정 방식 비교(E4)와 배치에 미치는 영향

동료 평가는 체크한 항목(좋은 점·아쉬운 점)과 서술 글로 이뤄진다. 항목만 세는 **규칙**, 감성 점수 API **Jev**, LLM 두 가지(서비스용 **OpenAI gpt-6-luna**, 사내 후보 **GLM-5.3-Flash**)를 같은 글에 돌렸다.

판정 방식	① 글로만 드러난 부정 (직설·혹평 34건)	② 돌려 말한 부정 (정답 있는 글 300건 중 부정 83건)	1,000건 비용	판단
규칙(체크한 항목만 셈)	4/34 (12%)	— (항목이 곧 정답인 시험)	0	글을 읽지 않아 글에 쓴 부정을 놓친다
Jev(감성 점수 API)	32/34 (94%)	14/83 (17%)	$0.02	노골적인 부정만 읽는다
GLM-5.3-Flash(사내 후보)	34/34 (100%)	43/83 (52%)	$0.09	완곡한 지적의 절반을 놓친다
OpenAI gpt-6-luna(서비스)	**34/34 (100%)**	**74/83 (89%)**	$0.08	**채택** — 정답과 상관 0.94

① 시연 원문 무작위 300건 중 "직설형 평가자"·"항목보다 독한 글" 34건. ② 항목대로 쓴 글에서 숨은 부정을 찾는 시험. 판정 기준은 측정 전에 커밋했다.

판정 차이는 배치 결정까지 바꾼다. 같은 시연 묶음을 규칙 판정과 AI 판정으로 각각 만들어 안 A를 서비스와 같은 설정(시드 4개·안 A 허용 차이 1%)으로 풀었다(`rehearsal/ai_effect.py`).

규모	글로만 드러난 부정 평가	그런 사이가 협업 보상 대상에 든 수 규칙 → AI	그런 사이를 같은 팀에 규칙 → AI	팀 협업 점수 규칙 → AI
100명	123건 / 1,193	28 → **0**	9 → **4**	6.29 → 7.68 (+22%)
200명	197건 / 2,231	28 → **0**	16 → **4**	15.34 → 16.38 (+7%)
300명	313건 / 3,495	23 → **0**	14 → **8**	17.24 → 19.19 (+11%)

규칙으로 판정하면 서로 나쁘게 쓴 사이가 "궁합 좋은 상위 200쌍"에 들어가 함께 배치될수록 점수를 받는다. AI 판정은 세 규모 모두 그런 사이를 보상에서 뺐다. 팀 협업 점수는 AI가 읽은 평가로 다시 채점한 값, 두 안 모두 위반 0(10-19 다시 잼, `rehearsal/results/ai-effect.json`).

3.6 그 밖의 생성형 AI 구성요소 (서비스 운영 모델: 외부 gpt-6-luna)

쓰임	측정	결과	비용·시간
교체 설명 (E5)	운영 중 교체 100건	바로 사용 **93%**, 새 위반 교체의 "보류" 준수 90%(어긴 글은 규칙 설명으로 대체), 지시문 강화 뒤 보류 18/20·인정 경력 출처 9/10	평균 3.5초, 100건 $0.06
인사팀 소명 글 (E7c)	사업 × 3회 843편(조율 데이터 618 + 처음 보는 데이터 225), 사실을 일부러 틀리게 한 오류 주입	검사 통과 **99.0%**(612/618), 처음 보는 데이터 **100%** (225/225). 주입 오류 **13,316/13,316** 검출, 채택 글 무결성 837/837	편당 약 8초·$0.0009
소명 요약 (10-20)	229건, 세 문장·자리표시 2~4개·불리한 사실 하나 이상을 검사	AI 요약 채택 **92.6%**, 걸리면 규칙 요약(7.4%), 무결성 실패 0(10-22 다시 잼)	229건 $0.12
기술 이름 사전	기술 이름 절반을 별칭으로 바꾼 사본	사전을 쓰면 점수 차이 0, 없으면 쌍의 86%가 바뀌고 배치 57.8 → 49.1	—

사내 LLM으로 바꾸면(사내 서버를 쓸 수 없어 GLM-5.3-Flash로 가정): 돌려 말한 부정 검출 89% → 52%, 교체 설명 바로 사용 93% → 97%·3.5 → 10.3초, 처음 보는 소명 채택 100% → 97.3%·8 → 60초 — 전환 때 더 큰 모델·대기 상한(40 → 300초) 조정이 필요하다. 서비스는 외부 AI로 운영하고(사용자 결정 10-11), 실데이터는 사내 주소이거나 관리자가 명시 허용했을 때만 AI를 부른다. 판정 기준은 측정 전에 커밋했다(사전 등록). 출처: `outputs/review-judge-comparison.html`, `rehearsal/results/ai-effect.json`, `outputs/briefing-llm-comparison.html`, `rehearsal/results/justify-scale-e7c.html`, `rehearsal/results/justify-summary.json`, `rehearsal/results/skill-dictionary.html`.

3.7 정확성 검증

층	방법	결과

자동 시험	`uv run --group benchmark pytest -q`, 느린 전 과정 E2E, 웹 단위 시험·타입 검사	2,140 passed · 1 skipped / E2E 21 passed / 웹 367 passed(10-23)
정답 대조 (Phase 0)	작은 사례를 사람 조합 완전 열거 + LP로 푼 정답과 비교 (제품 MILP 코드를 읽지 않는 별도 구현), 예산·가용률을 늘리면 점수가 줄지 않는지 등	13/13 통과(정답 대조 9/9 차이 0 — 전일 투입·동시 수행 허용 사례 2개 포함, 단조성 2/2 …). 모델을 바꿀 때마다 다시 돌림
솔버 비교 (Phase 1)	CBC·HiGHS·SCIP에 같은 동결 입력, 1스레드·케이스당 240초, 357칸(본평가 288칸 포함)	정답 사례 21건은 세 솔버 모두 통과. 나머지 솔버당 112건 중 시간 안 완료 HiGHS 79 · SCIP 75 · CBC 22(CBC는 상한을 내지 않아 품질 확인 불가) → HiGHS 채택
실행 중 독립 검증	솔버의 모든 반환 해를 원자료로 다시 계산(6.4)	통과한 해만 화면·PDF로. 불러온 편성 기록도 지금 규칙으로 다시 채점
리허설·재현	실제 서버로 100/200/300명 전 과정, 시연 대본 최종 리허설, GitHub에서 새로 받은 복제본으로 설치·빌드·전체 시험	대본 숫자와 화면 일치. 시간에 따라 갈리던 시험 1개를 찾아 고침
독립 코드 리뷰	변경마다 Codex 리뷰, 지적은 고치기 전 코드에서 실패하는 시험으로 고정	전체 감사 27건 수정(10-12), 최종 점검 2건 수정, 전일 투입 재설계 리뷰 4라운드(Codex 2 · Codex 한도 소진 중 Claude Opus 대체 2), 10-21~23 변경은 Codex 한도 소진으로 Claude 자체 리뷰

4. 정성적 성과

- **계산 결과를 얼마나 믿어도 되는지 함께 보여 준다.** 모든 결과에 "규칙 위반 없음 확인", "최선 증명" 또는 "최적값이 이 해보다 최대 몇 % 높을 수 있음", "시간 한도 도달·사용자가 멈춤·기록에서 연 결과"를 붙이고, 교체 점수는 "참고값 · 재최적화 아님"으로 밝힌다. PDF의 이 문구도 서버 서명이 맞을 때만 찍힌다.
- **생성형 AI를 "검사받는 부품"으로 썼다.** 결정은 최적화가, 읽기와 쓰기는 AI가 맡는다는 경계를 실험으로 정했다 — LLM 직접 배치(3.4)는 규모가 커지면 무너지고, 평가 글 판정(3.5)은 규칙으로는 안 된다. AI 글은 기계 검사를 통과해야 나가고(소명 글은 사실 번호와 연결 말만 고르고, 교체 설명은 인용을 원문과 대조), 걸리면 규칙 기반 글로 대신한다.
- **규칙을 측정으로 정했다.** 보상 쌍 수·익숙한 쌍 기준·감점 방식·솔버·소명 글 방식을 대안과 같은 데이터로 비교하고, 판정 기준을 측정 전에 커밋했다(7장).
- **부서장의 판단을 지우지 않고 남긴다.** 계산이 정하지 않는 것 — 등급 대체, 프로젝트 동시 수행 허용, 확정 취소 — 은 부서장이 사유와 함께 고르고, 데이터의 새 판(되돌리기 가능)과 인사팀 검토 자료의 잠긴 사실로 남는다. 계산 모델은 그대로 두어 부족 신호를 묻지 않는다.
- **측정한 대로 보고한다.** 사업 효과는 `NOT_CALIBRATED`로 표시하고, 가상 성과로 효과를 "입증"하는 것은 순환 검증이라 하지 않았다. 대안 C가 A보다 조금 높은 것처럼 직관에 어긋나는 결과도 그대로 둔다.
- **실제 도입을 염두에 둔 설계.** 실제 인사 자료와 같은 입력 계약·검증 리포트, 평가 원문을 화면·AI에 보내지 않고 항목 라벨만 쓰는 비공개 모드, 외부 AI 차단 규칙, 사내 LLM 전환 시 차이 측정(3.6).
- **AI로 개발 과정 자체를 운영했다.** Claude 두 계정과 Codex가 파일 소유 영역을 나눠 병행 개발하고(작업 분담표·인수인계 기록), 변경은 커밋 전에 독립 리뷰를 받았다. 리뷰 지적은 실패하는 시험으로 먼저 고정하고, 측정은 판정 기준을 먼저 커밋했다 — 12주 동안 커밋 약 610개, 시험 2,140개.

5. 시스템 구성

5.1 구성도

입력	**인사 시스템 API(기본) · CSV 묶음** 사람·단가표·기술 경력·업무 이력·월별 가용·사업·등급/기술 요구·동료 평가·현재 배치 등 14종 `core/ingest`	**엄격 검증·변환** 오류 파일·줄·칸, 경력 개월 → 대리 레벨, 기술 이름 사전 정규화·하위 기술 인정 `core/ingest/loader·skills`	
지식	**관계 그래프** 누가 누구와 몇 달 일했나, 익숙한 쌍 `core/graph/memory_graph`	**근거 색인** 동료 평가 원문·인정 경력 (SQLite) `core/graph/sqlite_store`	**지식 그래프** 사업·기술·사람 사실, 기술 지도 `core/kg`
계산	**점수** 기술 적합 S · 협업 C `core/scoring`	**MILP** HiGHS 시드 포트폴리오 · 대안 · 운영 중 K `core/optimize`	**독립 검증 · 평가기** 원자료로 재계산, 모든 배치를 같은 기준으로 채점 `validation · core/evaluate`
AI	**평가 글 판정** 극성 → C `api/review_judge`	**교체 설명(RAG)** + 인용 원문 대조 `api/rag/briefing`	**소명 글(GraphRAG)** + 사실·문법 검사 `core/kg/justify`

서비스·화면	**FastAPI** 최적화(SSE)·교체·운영 중·소명·작업 데이터 판(등록·확정 미리 계산·확정·취소)·검토 자료·설정, 결과 서명 `api/`	**웹** React · TypeScript `web/`	**PDF** 서명 확인 → 서버 재계산 → Chromium 출력 `api/pdf`

흐름: 입력이 검증을 통과하면 활성 데이터가 바뀌고, 점수·그래프를 만든 뒤 MILP를 푼다. 솔버의 해는 독립 검증을 통과해야 API가 내보내고, AI 글은 각 검사기를 통과해야 내보낸다(실패 시 규칙 기반 글).

5.2 기술 스택

영역	사용 기술	영역	사용 기술
언어	Python 3.12 · TypeScript 6	API	FastAPI, 서버 전송 이벤트(SSE)
최적화	PuLP + HiGHS(highspy), 비교용 CBC·SCIP	AI	OpenAI 호환 API — 외부 gpt-6-luna, 사내 가정 GLM-5.3-Flash
데이터 계약	Pydantic 2, CSV 묶음(zip) + manifest	웹	React 19 · Vite 8 · Tailwind 3
저장	인메모리 그래프, SQLite 근거 색인, 서버 데이터 폴더(업로드·교체 이력·서명 키)	보고서·시험	Playwright(Chromium) PDF · pytest · Vitest

5.3 신뢰 경계

- **결과 서명**: 계산 결과마다 서버 서명(`plan_token`)과 계산 신뢰도 서명을 붙인다. 교체·소명 글·PDF는 서명이 맞는 결과만 받고, PDF 지표는 서버가 명단으로 다시 계산한다.
- **AI 경로**: 실데이터 모드에서는 사내 주소이거나 관리자가 명시 허용했을 때만 AI를 부르고, 평가 원문은 화면·AI에 보내지 않는다.
- **접근**: 앱 전체 로그인 — 로그인하지 않으면 모든 API가 401(HttpOnly·SameSite=Strict 세션, 5회 실패 시 1분 잠금). 시연 기본 계정은 로컬 주소에서만 뜨고, 외부 주소는 비밀번호를 정해야 서버가 시작한다.
- **자원 보호**: 새 계산 동시 2개(초과 429), 시간 한도 상한 900초, 요청 본문 2 MiB, 연결이 끊기면 시드 프로세스까지 정리, PDF 브라우저는 외부 요청 차단.

6. 주요 적용 기술

6.1 배치 최적화 모델 (혼합정수계획, MILP)

목적(최대화)	$\Sigma_{i,j}\, S_{ij}\cdot z_{ij} + \lambda\cdot\Sigma_{(p,q)\in P,\, j}\, C_{pq}\cdot y_{pqj} - \mu\cdot\Sigma_{(p,q)\in F,\, j}\, y_{pqj} - 100\cdot\Sigma_{j,g}\, \mathrm{slack}_{jg}$ $(\lambda = 0.3,\ \mu = 0.2)$
전일 투입 자격	$z_{ij} = 0$ (사업 j 기간 중 가용률$_{im}$ < 1인 달이 있으면 — 육아휴직 등)
프로젝트 동시 수행	$\Sigma_{j:\, m\in j}\, z_{ij} \le$ 동시 수행 상한$_i$ (달 m마다. 기본 1, 부서장이 사유와 함께 허용한 사람만 2)
월 예산	Σ_i 단가$_i\cdot z_{ij} \le$ 예산$_j$ (동시 수행자는 사업마다 전액)
등급 정원	$\Sigma_{i\in g}\, z_{ij} + \mathrm{slack}_{jg} =$ 요구 인원$_{jg}$ (사업에 적힌 등급 g만)
쌍 선형화	$y_{pqj} \le z_{pj},\ y_{pqj} \le z_{qj},\ y_{pqj} \ge z_{pj} + z_{qj} - 1$

- 변수: `z` 배치 여부(0/1 — **전일 투입**이라 투입률 변수가 없다, 예전 투입률 방식은 설정에 남김), `y` 두 사람이 같은 사업에 있는지(곱의 선형화), `slack` 등급 정원 미충원(큰 감점, 결과에 노출). 탐욕 해를 초기해로 넣는다.
- **S**(기술 적합) = 요구 기술마다 min(보유 레벨 ÷ 요구 레벨, 1)을 중요도(1~5)로 가중 평균. 원천에 숙련 레벨이 없어 경력 개월을 대리 레벨로 바꾼다(경계 12·36·60·96개월). **C**(협업) = 0.4·min(함께한 개월 ÷ 12, 1) + 0.6·동료 평가 점수(평가 글 극성은 LLM 판정).
- **P** = |C| 상위 200쌍만 보상한다(5,000쌍이면 200명에서 600초에도 빈자리 57석). **F** = 익숙한 쌍(최근 36개월 중 12개월 이상 같은 사업)을 감점해 오래 붙어 다닌 사이만 몰리는 팀을 흩는다 — 지금 같은 진행 사업에 함께 있는 두 사람은 그 사업에서 감점하지 않는다(사용자 결정 10-19).
- 필수 기술은 하드 제약이 아니라 S로 유도하고(실제 자료 답변 "선호"), 정원에 적히지 않은 등급은 예산·가용률 안에서 자유롭게 고를 수 있다.

6.2 대안과 운영 중 증분 최적화

- **대안 B~D**: 앞선 안과 (사람, 사업) 겹침 ≤ 80%인 다양성 컷 + 품질 하한(A − 5%). 빈 팀·중복·A보다 빈자리가 많은 후보는 버리고, 없으면 "조건을 만족하는 대안 없음"으로 억지로 채우지 않는다.

- **운영 중 K**: 기존 배치를 그대로 두고 사람마다 0/1 이동 변수를 두어 Σ ≤ K. 옮긴 사람은 "원래 없던 사업"에 들어가야 한다(빼서 놀리는 해 차단), 잠근 사람은 옮기지 않는다. 빈자리 감점(자리당 100)이 섞인 목적에서 5% 허용 차이는 1~2명 이동의 개선(1~2점)을 통째로 가리므로 **허용 차이 0**으로 풀어 최선을 증명한다.
- **확정과 등급 대체**: 모든 확정은 원천 CSV 묶음을 고쳐 업로드와 같은 검증으로 켜는 새 판이고, 새로 생기거나 커진 위반·고정 배치 빼기를 거절한다. 손 편성이 **새로 넘친 등급 × 모자란 자리**를 만들면 가까운 등급부터 짝지어 정원을 바꾸고(사유 필수, 넘친 예산 증액), 미리 계산·What-if 채점·확정 검증이 같은 함수를 쓴다. 다른 사업에서 빼 와 생긴 빈자리는 같은 확정에서 **메우거나 "메우지 않음"**(사유, 그 정원을 줄임)을 정해야 확정된다. 연간 계획은 고정이 있으면 계산 전에 유지·풀기를 묻는다.

6.3 솔버 운영

- **시드 포트폴리오**: 같은 모델을 시드만 바꾼 HiGHS 프로세스 4개로 동시에 풀어 가장 좋은 해를 고른다(1시드면 조밀한 데이터에서 대안이 품질 하한에 걸려 안 A만 나오기도 했다). 모델은 시작 뒤 전용 파이프로 넘긴다 — 시작 인자로 넘기면 자식이 시작 중 죽을 때 부모가 멈췄다(실측).
- **증명 상한**을 함께 기록해 "최적값이 최대 몇 % 높을 수 있음"으로 보여 준다. 전일 투입 권장 한도 100명 30초·200명 60초·300명 90초, 12개월은 ×4(측정 기반, 관리자 설정).
- **계산 멈추기**: 사용자가 멈추면 풀던 안은 그때까지의 해로 끝나고(종료 사유 "사용자가 멈춤", 시간 한도와 구분) 다음 안은 시작하지 않는다. 연결이 끊기면 즉시 중단한다.
- **편성 기록**: 한 번의 편성(원천 CSV · 평가 글 판정값 · 계산 설정 · 편성안 · 운영 중 K 비교)을 JSON 하나로 내보내고 다시 불러온다. 불러올 때 업로드와 같은 검증으로 다시 만들고 지금 평가기로 다시 채점해 위반이 있으면 받지 않는다. 기록이 주장하는 솔버 증명은 버린다 — 화면·PDF는 "위반 없음 · 기록에서 연 결과", 증명은 다시 계산해야 붙는다.

6.4 독립 검증기와 평가기

- 솔버가 낸 원시 해를 제품 MILP 코드와 별개로 다시 계산한다: 결측·NaN·범위·0/1, 월별 가용률·월 예산·등급 정원·동시 사업, 쌍 범위, 목적 4항, 그리고 화면에 내보낼 명단. 검증할 수 없는 값을 0으로 채우지 않는다.
- HiGHS는 z = 1.0000000000000007 같은 값을 돌려준다 — 1e-9 안쪽의 경계 잔차만 경계로 옮기고(원본 보존), 예산의 극미세 초과(상대 1e-7 이하)는 보정 LP 뒤 다시 검증한다. 이 처리 없이는 HiGHS 해가 전부 "검증 실패"였다(실측).
- 평가기는 어떤 배치든 현행 목적·제약 전체로 다시 채점한다 — 교체 검토·What-if(참고값), 확정 미리 계산, 단순 규칙 비교, 편성 기록 확인, PDF가 모두 같은 잣대를 쓴다.

6.5 검사받는 생성형 AI

쓰임	AI가 하는 일	기계 검사	실패하면
평가 글 판정	동료 평가 글의 극성(긍정/부정)을 판정 → 협업 점수 C에 반영. 판정값은 데이터 버전에 포함	출력 형식·범위	평가 항목 점수 사용
교체 설명 (RAG)	근거 색인의 동료 평가·점수 변화로 설명, 평가를 출처 번호와 함께 인용	인용이 원문과 한 글자도 다르지 않은지, 출처가 있는지, 위반 교체면 첫 문장이 보류인지, 인정 경력 출처 표기	규칙 기반 설명
소명 글 (GraphRAG)	지식 그래프 사실 목록 `{F#}` 중 쓸 것을 고르고 정해진 문법(주어·라벨·연결 말·맺음말)으로만 엮음. 숫자·충족 여부는 서버가 채움	없는 자리표시, 허용 목록 밖 연결 말, 팀원 아닌 ID, 사실과 라벨의 짝, 사람별 사실 누락. 빠뜨린 불리한 사실은 서버가 덧붙임	정해진 틀의 글
상황을 말로·기술 이름 정리	말·이름을 사전 개념 id로 옮김(사전 목록 안에서만). 사람 정보는 보내지 않음	id가 사전에 있는지, 근거 말이 원문에 있는지, 사람 특성 말이 근거인지, 헷갈리는 짝(Java↔JavaScript 등) 표시. 반영은 사람이 고른 것만	사전 별칭 찾기·문자열 후보

소명 글은 처음에 자유 문장 + 금지어 목록으로 시작했으나 검토 때마다 뚫려, "AI는 사실 번호와 정해진 연결 말만 고르고 문장의 사실 부분은 서버가 채운다"로 바꿨다(E7b 94.7% → E7c 99.0%).

6.6 지식 그래프와 기술 이름 사전

- **기술 이름 사전**(개념 228개): "postgres"·"포스트그레스"·"PostgreSQL 14"처럼 갈린 이름과 버전 표기·괄호 병기를 한 이름으로 묶는다. 사전에 없는 이름은 합치지 않고 경고만 한다.
- **하위 기술 부분 인정**: 하위 기술 경력의 $0.7^{깊이}$를 상위 기술에 인정(Spring → Java)하고, 인정분은 출처·개월을 따로 남겨 본인 경력과 구분한다. 사전 버전·인정 비율은 계산 버전에 들어간다.
- **지식 그래프**(`core/kg`): 사업·요구 기술·팀원·협업·후보 비교·규칙 준수를 사실로 만들어 소명 글에 쓰고, 기술마다 공급·수요를 모아 조직 기술 지도를 만든다. 저장 기술은 측정으로 골랐다(Neo4j 우위 0/20 → 제거, RDF 추론은 가중 부분 인정을 표현하지 못해 인메모리 그래프 유지).

7. 주요 설계 판단과 근거

판단	비교한 대안	근거(측정)	결정
서비스 솔버	CBC · SCIP · HiGHS	솔버당 112건 중 시간 안 완료 22 · 75 · 79(CBC는 품질 확인 불가). 규모 리허설에서 CBC는 세 규모 모두 최선 1% 안에 못 듦	HiGHS 고정
협업 보상 쌍 수	5,000쌍	200명 600초에도 빈자리 57석, 300명 해 없음 → 200쌍으로 모든 규모 빈자리 0	|C| 상위 200쌍
"익숙한 쌍" 기준	10년 중 6개월 · 5년 중 12개월 · 3년 중 12개월	300명 익숙한 쌍 11,733 · 1,352 · 395개. 10년 기준은 200명에서 빈자리 79석	최근 3년 중 12개월
오래 붙어 다닌 사이	쌍 감점 · 새 동료 수 하한 (문헌의 파트너 다양성)	하한은 SI 팀 크기에서 거의 걸리지 않고 200명 풀이를 망침. 감점은 익숙한 쌍을 200명 29 → 6, 300명 46 → 5로 흩고 품질 손실 −3.2% · −0.05%	쌍 감점 0.2 유지
투입 방식	투입률 나누기 · 동시 수행 인원 상한 · 감점 · 전일 투입	나누기는 잘 풀수록 쪼개기가 늘고, 상한·감점은 쪼개기를 못 막았다. 전일 투입은 1분 안 최선 증명(3.1)	전일 투입 + 사람 단위 허용 (1.3)
AI의 역할	LLM 직접 배치	100명에서 규칙 위반 또는 솔버의 55%, 6~11분(3.4)	결정은 솔버, AI는 읽기·쓰기
평가 글 판정	규칙(항목만) · Jev · GLM-5.3-Flash · OpenAI	규칙은 글에 쓴 부정을, 감성 점수 API는 돌려 말한 지적을, 사내 후보는 그 절반을 놓친다. 규칙이면 나쁘게 쓴 사이가 협업 보상에 든다(3.5)	LLM(OpenAI) 판정
소명 글 방식	정해진 틀(템플릿) · GraphRAG	근거 오류는 둘 다 0, 읽기 쉬운 쪽으로 — 템플릿은 실패 시 대체 글로 유지	GraphRAG
숙련도 표현	경력 개월 비율 직접 사용	가상 데이터로는 우열을 가릴 수 없고 바꾸는 비용이 큼	대리 레벨 유지
자리와 다른 등급을 넣을 때	확정 막기 · 정원을 말없이 늘리기 · 대체 + 사유	막으면 현장 관행(중급 대신 초급, 중급이 없어 고급)을 못 담고, 말없이 늘리면 부족 신호와 판단 근거가 사라진다	사용자 결정: 위·아래 허용, 사유 필수, 넘친 예산 증액
What-if의 기준	계산한 안 A~D · 지금 실제 배치	부서장의 질문은 "지금 배치에서 이렇게 바꾸면"이다	연간 계획 ②와 분리

각 결정은 사용자 확인을 거쳤다. 측정 문서는 `rehearsal/results/·docs/demo-notes.md·docs/skill-score-decision.md`. 하위 기술 인정 70%는 근거 데이터가 없는 가정값이다.

8. 사업적 가치와 확장

8.1 누가, 얼마나 자주 쓰는가

장면	쓰는 사람	빈도	이 시스템이 바꾸는 것
연초 계획	인력 운영 담당·사업부장	연 1~2회	조건을 모두 지키는 안 4개를 10분 안에 비교(300명 4.2분 · 연간 100명 5.4분), 최선과의 거리까지 표시
신규 사업 편성·긴급 투입	부서장·인력 운영 담당	신규 발주·이슈 때마다 (수시)	"몇 명만 옮기면 되나"를 K=0~5 모두 최선 증명으로, 300명도 전체 약 66초(K 하나당 34초 안, 3.2)
What-if·교체 검토	부서장·사업 책임자 (PM)	수시	지금 배치에서 교체 전후 점수·위반·근거 인용 설명을 몇 초 안에, 담은 변경을 한 번에 확정 → 검토 자료
인사팀 검토 자료	부서장 → 팀장·인사팀	확정마다 사업 수만큼	잠긴 시스템 사실(원래 사업 영향·메움·등급 대체 사유) + 부서장 판단·합의 + 해명·이의 → PDF

8.2 비용 구조(측정값)

- **솔버 라이선스 0원** — 오픈소스 HiGHS로 300명 안 A를 최선 대비 0.9% 안에서 푼다(92초). 상용 솔버 무료판은 변수 2,000개 제한이라 100명 모델도 넘는다.
- **AI 호출 비용은 무시할 수준** — 300명 조직의 평가 글 4,012건을 처음 판정하는 데 약 $0.32(이후 저장값 재사용), 사업 60건 소명 글 전체 약 $0.05, 교체 설명 한 건 약 $0.0006.
- **서버 한 대** — 개발용 데스크톱 한 대에서 300명 계산·PDF까지 돌렸다.

8.3 효과 추정 — AI를 쓸 때와 아닐 때의 사람 시간

측정값이 아니라 공개 연구·실측 속도에 가정을 더한 **추정**이다(근거 등급 표시) — 실제 도입 때 검증할 가설로 둔다. 300명 조직 예.

일	AI 없이(사람)	AI를 쓰면	줄어드는 몫	근거
평가 글 4,012건 읽고 판정	건당 15~75초(중심 30초) → 약 33시간	판정은 기계 약 10분 + 5% 표본 확인 1.7시간	약 95%	한국어 묵독 분당 550음절(쉬운 단문 상한)【관찰】, 짧은 글 표시 중앙값 31.6초【관찰】, AI 1,372건 201초【실측】
"왜 이 팀인가" 한 편	근거 모으기·쓰기 30~90분【가정】	8초 생성 + 사실 확인	37~40%(보수 25%)	전문직 453명 무작위 실험: 글쓰기 27분 → 17분, 품질 +18%(Noy & Zhang, *Science* 2023)【통제】
긴급 상황 → 요구 입력	정리·사전 검색 5~15분【가정】	읽기 3초 + 확인 1~3분	AI 몫 2~14분	정확도는 실측(3.3). 후보 검토 25~209분 → 11~43분의 대부분은 기존 계산(순위·최선 조합) 몫이라 따로 적는다
발주 자료 한 편 → 사업 정보·필요 기술 (제안요청서 1.7만 자 예)	정독 20~36분 + 기술 고르기·양식 10~20분【가정】→ 약 30~55분	AI 읽기 약 30초 + 근거 보고 확인 5~10분【가정】	약 70~85%	묵독 분당 550음절(쉬운 단문 상한)【관찰】 ~300음절(기술 문서 정독)【가정】× 이 자료 한글 10,710음절, AI 시간은 실측(3.3)
낯선 기술 이름 300개	건당 61초 → 5.1시간	후보 3개에서 고르기 38초 → 3.2시간	약 37%	직업 분류 코딩 수작업 60.8초 → 도구 보조 38초 (OPERAS)【관찰】, 후보 3개 안 정답 100%(3.3)

최적화 쪽(가정): 연초 계획 72 → 8시간, 편성·긴급 투입 6 → 1시간. 시간보다 큰 효과는 결정의 질이다(3.1·3.5). 한국어 업무의 AI 단축률을 직접 잰 통제 연구는 찾지 못했다.

8.4 차별점과 도입·확장 경로

- **차별점**: ① SI의 등급 정원·월 예산·휴직·협업 이력을 현업 규칙(전일 투입·사람 단위 동시 수행·등급 대체) 그대로 하나의 최적화와 확정 규칙으로 풀고, 증명 상한과 독립 검증으로 "얼마나 믿어도 되는지"까지 보인다. ② 가장 잦은 "최소 인원 이동 재편성"을 최선 증명으로 푼다. ③ AI는 검사받는 부품이라 설명 글을 자동화하면서도 근거를 대조한다.
- **실제 도입 시 단계**(PoC 범위 밖): 섀도 운영(실제 편성과 나란히 3개월, 빈자리·예산 초과·재배치·고객 교체 요청·편성 시간 기록) → 한 사업부 파일럿 → 확대. 그때 할 일은 인사 시스템 연결 설정(받기 경로는 있고 가짜 인사 시스템으로만 시험), 사내 LLM 연결, 계정·권한·감사·배포다.
- **결재 연결**(실제 도입 시): 검토 자료를 **사내 전자결재**에 올리고, 팀장·인사팀은 같은 화면을 읽기 전용으로 보며 승인·반려만 한다. 배치는 마지막 승인 때 바뀐다(이 PoC의 확정은 바로 반영하는 단순화).
- **효과 입증 경로**(실제 도입 시): 과거 성과 점검 도구(`core/evaluate/outcome_check.py`)와 계산 버전 관리로, 성과가 쌓이면 계수를 보정한다.
- **확장성**: 입력 계약·평가기·검증기·지식 그래프는 업종에 묶이지 않아, 사람을 프로젝트 단위로 배치하는 조직(컨설팅·엔지니어링 등)에 같은 구조로 쓸 수 있다.

9. 한계와 다음 단계

- **사업 효과는 실제 성과 자료로 확인하기 전이다**(`NOT_CALIBRATED`) — 모든 수치는 가상 데이터와 설계한 점수표 안의 값이고, 실제 도입 시 섀도 운영(8.4)으로 확인한다.
- 점수 계수(협업 0.3·익숙한 사이 0.2·하위 기술 70%·경력 레벨 경계)는 설계값이고 평가 글도 가상 글이다 — 실제 성과·실제 글로 다시 잰다(계산 버전과 평가기를 분리해 둠).
- 등급 대체·"메우지 않음"은 확정 때 정원 표를 바꾸는 손 편성 규칙이다(자동 계산은 빈자리로 보인다). 정원이 사업 전체 기간 단위라 몇 달만 비워 두기는 아직 없다.
- AI 전후 측정은 가상 데이터·문장이고(요청 48문장은 구간이 넓다), 8.3의 시간은 추정이다. 발주 자료의 사업 정보 채우기는 AI가 앞 4만 자만 읽고(사업명·기간·예산은 앞머리에 있다), 필요 기술 읽기·요약은 20만 자까지 나눠 읽는다. 요약 검사는 근거 없는 항목을 거를 뿐 모두 막지는 못한다. 월 예산 환산(1점 = 1만 원)은 가정이다.
- 운영 기능은 시제품 수준(관리자 하나·단일 서버)이고, 전일 투입은 가용률 0 또는 1을 전제로 한다.

부록. 재현 방법과 근거 파일

```
uv sync && scripts/run_poc.sh   # 실행 → http://127.0.0.1:8000 · 시험: uv run --group benchmark pytest -q (2,140 passed)
```